TRASTERO WARS

Subastas de trasteros en tiempo real.
2–5 jugadores, 20 rondas y una ruleta para remontar.

TypeScript | Phaser 3 | Node + ws | Pixel art 480×270



Vista previa: solo siluetas y pistas ambiguas. El valor se descubre al revelar.

"Un Storage Wars de navegador: cinco desconocidos, un trastero y doce segundos para decidir."

EL JUEGO

Subastas a ciegas, decisiones de 12 segundos

Cada trastero es una caja negra: pagas por lo que crees que hay dentro.

2–5	20	1.000 €	22
jugadores por sala	rondas por partida	dinero inicial	objetos distintos

PUJAS A CIEGAS

Solo ves siluetas, polvo y pistas ambiguas. Dos objetos pueden parecer idénticos y valer 10× más uno que otro.

REVELADO

Quien gana paga su puja y suma el valor real del lote. El beneficio (o la ruina) se calcula en público.

RULETA

Tras las rondas 5, 10 y 15: 20 segundos para apostar en rojo, negro o verde. La vía de remontar… o de hundirte.

Sin cuentas, sin matchmaking, sin chat libre: entras con un apodo y un código de sala.

De la sala al ranking en seis pasos

Todo ocurre en el servidor; el cliente solo dibuja lo que le llega.

01 Sala y listos

Código de 4–6 caracteres. Apodo, listo, empezar. 2–5 jugadores.

02 Se abre el trastero

Lote de 2–5 objetos. Siluetas iguales para todos y pistas ambiguas.

03 Subasta

12 s de temporizador. Pujas al alza con incremento mínimo de 10 €.

04 Revelado

Nombre y valor de cada objeto: total, pujado y neto del ganador.

05 Ruleta

Tras 5, 10 y 15. Opcional, 20 s, rojo/negro 1:1 y verde 35:1.

06 Ranking

Al acabar la ronda 20 gana quien tenga más dinero.

El cliente nunca decide pujas, timers, orden de turnos ni resultados.

Doce segundos, tres prórrogas y ni un euro de más

Subasta ascendente: nadie puede pujar contra sí mismo.

Puja mínima 10 €

El primer postor abre en 10 €; después, cada puja suma al menos 10 €.

Botones +10 / +25 / +50

Además de la puja mínima y de “pujar todo”, para no teclear en plena subasta.

12 s por puja

Cada puja reinicia el reloj. El servidor es el único que lo mide.

Máx. 3 prórrogas de 3 s

Pujar en el último instante alarga, pero solo tres veces: el lote siempre cierra.

Lote desierto

Si nadie puja, no se mueve dinero y se registra el trastero como desierto.



La prórroga existe para que ganar no sea cuestión de reflejos: quien puja en el último segundo no congela el lote.

REGLAS · INFORMACIÓN

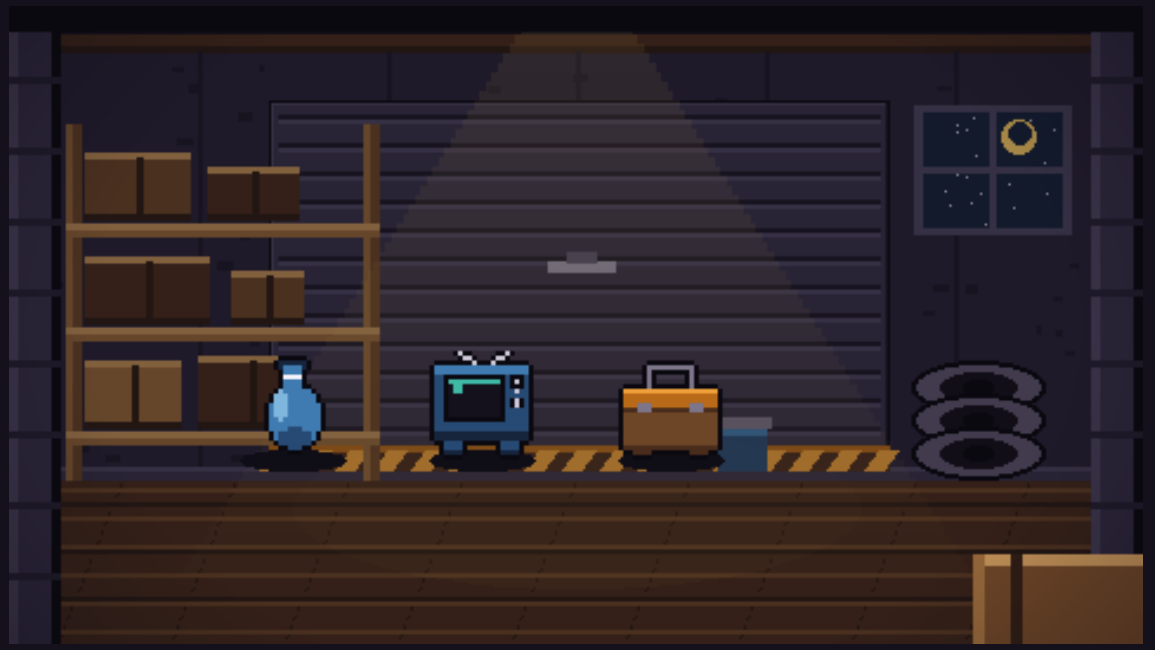
Lo que ves antes… y lo que hay dentro

La misma silueta puede esconder una joya o un trasto: el valor nunca viaja al cliente antes del revelado.



ANTES · siluetas + pistas

Pistas ambiguas: «pesa como un muerto», «el dial gira». Nunca dicen precio ni rareza.



DESPUÉS · objetos y valores

Total del lote, puja ganadora y neto. El historial guarda cada trastero de la partida.

Regla dura: los valores reales viven en el servidor. Un test y un escáner del bundle fallan si aparecen en el cliente.

La ruleta: 20 segundos para remontar

Apuestas opcionales tras las rondas 5, 10 y 15, con commit–reveal verificable.

RONDA

1	2	3	4	**5**	6	7	8	9	**10**	11	12	13	14	**15**	16	17	18	19	20

En las rondas 5, 10 y 15 se abre la ruleta

20 s de ventana

Eliges cantidad y color, o pasas. Si nadie gira, el servidor gira sola al agotarse el tiempo.

Rojo / negro 1:1

Aciertas y ganas lo apostado; fallas y lo pierdes. Apuesta mínima 10 €, máxima tu dinero.

Verde (0) 35:1

La apuesta arriesgada. Probabilidad baja, pago alto: la vía real de remontar.

Commit–reveal

Antes se publica sha256(semilla). Al girar se revela la semilla y el resultado se recalcula en público.

El resultado no es un número “de la casa”: sale de la semilla comprometida, así que cualquiera puede auditarlo.

STACK

Lenguajes y herramientas

Un monorepo TypeScript, un pipeline de arte en Python y despliegue con Docker.

Tecnología	Para qué	Volumen
TypeScript	Cliente, servidor, reglas, protocolo y tests	≈ 9.600 líneas · 61 ficheros
Python + Pillow	Pipeline de arte: sprites, paleta, fuente bitmap, SFX	≈ 2.200 líneas
JavaScript / MJS	Simulador de balance y herramientas E2E	≈ 430 líneas
CSS / HTML	HUD accesible sobre el canvas	≈ 400 líneas
YAML / Docker	CI en GitHub Actions y despliegue con Caddy + TLS	CI en cada PR

LIBRERÍAS PRINCIPALES

Phaser 3.90
motor de escenas y sprites

Vite 8
build del cliente

Node 22 + ws
servidor autoritativo

zod 3
validación del protocolo

Vitest 2
tests y cobertura

Turborepo 2 + pnpm 9
monorepo y caché

DEMO

De cero a jugar en menos de cinco minutos

Dos pestañas, un código de sala y a pujar.

ARRANCAR

```
pnpm install
pnpm build:packages
pnpm --filter @trastero/client run build
pnpm start
```

http://localhost:2567
(dos pestañas, mismo código)

SIN SERVIDOR

```
pnpm --filter @trastero/client run dev
```

?mock=1
3 rondas con bots y timers reales
?mock=1&roulette=1

DESPLIEGUE

```
docker compose up --build -d
```

App + Caddy
TLS automático
un comando, sin dependencias

Partida corta para probar: GAME_ROUNDS=2 · La misma imagen se usa en CI para el smoke E2E.

Nada de instalar base de datos: el estado de las salas vive en memoria y el servidor sirve el cliente compilado.

CIERRE

Qué demuestra Trastero Wars

Un juego completo, con reglas auditables y una ingeniería que se puede enseñar.

Jugable de principio a fin

Lobby, 20 rondas, revelado, ruleta, ranking y reconexión.

Reglas en un solo sitio

game-core puro, con reloj y RNG inyectados: testeable y reutilizable.

Contrato de red tipado

22 mensajes zod, deduplicación y validación en el servidor.

SIGUIENTES PASOS

1. Ajustar la curva de valor con el simulador de 1.000 partidas.
2. Generar los lotes desde el catálogo de arte (sin ids duplicados).
3. Reconciliar la UI DOM de D con el cliente de escenas de C.

GRACIAS

¿Jugamos una ronda?

Trastero Wars · Hackathon

github.com/Tuco74/Hackathon-etse